

SE 3242: Android Application Development

Week 3: Kotlin Essentials

Scope Functions, Generics, Delegation, Introduction to Jetpack Compose



Engr. Daniel MOUNE

`moune.daniel@ictuniversity.edu.cm`

ICT University, Cameroon Campus, Yaoundé

P.O. Box 526, 1 Avenue Dispensaire Messassi, Zoatoupsi

Imparting ICTs in all academic disciplines

Quick Recap – OOP in Kotlin

Classes & Inheritance

- ▶ Primary/secondary constructors, `init` blocks
- ▶ open classes and methods
- ▶ Abstract classes
- ▶ Interfaces (multiple inheritance, default methods)

Data & Sealed Classes

- ▶ data class – auto-generated methods
- ▶ `copy()`, destructuring
- ▶ Sealed classes for restricted hierarchies
- ▶ Object declarations (singletons) and companion objects

Project Milestone 3

Your OOP domain model is due next Monday – make sure to include inheritance, interfaces, and at least one data class.

“Now we dive into more advanced Kotlin features that make code concise and expressive.”

Today's Roadmap – Scope Functions, Generics, Delegation

Part 1: Scope Functions (40 min)

- ▶ let, apply, run
- ▶ with, also
- ▶ Differences and use cases

Part 2: Generics Basics (25 min)

- ▶ Generic classes and functions
- ▶ Type parameters and constraints

Part 3: Generics – Variance (25 min)

- ▶ Covariance (out)
- ▶ Contravariance (in)
- ▶ Type projections

Part 4: Delegation (20 min)

- ▶ Class delegation (by)
- ▶ Property delegation (lazy, observable)

Part 5: Mini-Exercises (10 min)

“These features will make your Kotlin code more elegant and reusable.”

Kotlin vs Dart – Scope Functions (Part 1)

Kotlin – let

```
val result = "Hello".let {  
    println(it)           // prints "Hello"  
    it.length             // returns length  
}  
// result: 5
```

let is often used for null checks:

```
name?.let { println("Name is $it") }
```

apply and run

```
val person = Person().apply {  
    name = "Alice"    // modify receiver  
    age = 25  
}
```

```
val result = run {  
    // independent block  
    println("Computing...")  
    42  
}
```

Dart – no direct equivalent

Dart doesn't have built-in scope functions, but you can use cascade notation or extensions:

```
var result = () {  
    print('Hello');  
    return 'Hello'.length;  
}(); // immediately invoked closure
```

Or use the let method from packages like dartz.

Kotlin vs Dart – Scope Functions (Part 2)

Kotlin – with and also

```
val numbers = mutableListOf(1, 2, 3)
with(numbers) {
    add(4)
    // operate on receiver
    removeAt(0)
}

val result = numbers.also {
    it.add(5)
    // also returns receiver
}.size
```

Dart – cascade notation ..

Dart's cascade (..) provides similar functionality:

```
var numbers = [1, 2, 3];
numbers
  ..add(4)
  ..removeAt(0);

// also-like behavior
var result = numbers..add(5);
print(result.length);
```

Summary of Kotlin Scope Functions

Function	Receiver	Return
let	it	lambda result
apply	this	receiver
run	this	lambda result
with	this	lambda result
also	it	receiver

Generics – Reusable Code for Multiple Types

Generic Class

```
class Box<T>(var value: T) {  
    fun get(): T = value  
}  
  
val intBox = Box<Int>(42)  
val stringBox = Box("Hello") // type inference
```

Generic Function with Kotlin

```
fun <T> singleton(item: T):  
List<T> = listOf(item)  
  
val numbers = singleton(5) // List<Int>
```

Dart Equivalent

```
class Box<T> {  
    T value;  
    Box(this.value);  
    T get() => value;  
}  
  
T singleton<T>(T item) => [item];  
// Very similar syntax.
```

“Generics allow type-safe code without sacrificing flexibility.”

Generic Constraints – Upper Bounds

Upper Bound with :

Restrict a type parameter to a specific supertype.

```
fun <T : Number> sum(list: List<T>): Double {  
    return list.sumOf { it.toDouble() }  
}  
  
sum(listOf(1, 2, 3))           // OK (Int is Number)  
// sum(listOf("a", "b"))      // Error: String is not Number
```

Multiple Constraints (where clause)

```
fun <T> ensure(item: T)  
where T : CharSequence,  
T : Comparable<T> {  
    // item has both interfaces  
}
```

Dart Equivalent

```
T sum<T extends num>(List<T> list) {  
    return list.reduce((a, b) => a + b);  
}
```

Dart uses extends for upper bounds.

Variance – Covariance (out)

The Problem

```
open class Animal
class Dog : Animal()

// Without variance:
Box<Dog> is not a subtype of Box<Animal>
```

We want Box<Dog> to be usable where Box<Animal> is expected (if we only read from it).

out (covariant) – producer

```
class Box<out T>(private val value: T) {
    fun get(): T = value
}

fun takeAnimalBox(box: Box<Animal>) {
    val animal = box.get()
}

val dogBox = Box(Dog())
takeAnimalBox(dogBox) // OK - Box<Dog> is subtype of Box<Animal>
```

out means T can only appear in output positions (return types).

“Covariance makes generic types more flexible when they produce values.”

Variance – Contravariance (`in`)

`in` (contravariant) – consumer

When you only consume (accept) values of type `T`.

```
class Box<in T> {  
    fun put(item: T) { ... }  
    // fun get(): T // not allowed with 'in'  
}  
  
fun putDog(box: Box<Dog>) {  
    box.put(Dog())  
}  
  
val animalBox = Box<Animal>()  
putDog(animalBox) // OK - Box<Animal> is subtype of Box<Dog> for consumption
```

`in` allows a supertype to stand in for a subtype when writing.

Dart Variance

Dart does not have declaration-site variance annotations like `in/out`. Instead, it uses use-site variance (like Kotlin's type projections) with `extends` and `super` constraints.

Delegation – Reuse via Composition

Class Delegation (by)

Implement an interface by delegating to another object.

```
interface Sound { fun makeSound() }  
class Scream : Sound { override fun makeSound() = println("AAARGH!") }  
  
class Person(sound: Sound) : Sound by sound  
// Person automatically forwards makeSound() calls to sound
```

Property Delegation – lazy

Initialize once on first access.

```
val expensive: String by lazy {  
    println("Computing...")  
    "result"  
}
```

observable

Observe changes.

```
import kotlin.properties.Delegates  
  
var name: String by Delegates.observable("in"  
    prop, old, new -> println("$old -> $new")  
}
```

“Delegation reduces boilerplate and encourages composition over inheritance.”

Type Projections – Use-Site Variance

The Problem

Sometimes you need variance only at a specific use, not throughout the whole class.

```
fun copy(from: Array<Any>, to: Array<Any>) { ... }  
// Can't pass Array<Int> as from, even if we only read
```

out Projection (covariant)

```
fun copy(from: Array<out Any>, to: Array<Any>) {  
    // from: only safe to read (producer)  
}  
// Now Array<Int> is acceptable
```

in Projection (contravariant)

```
fun fill(dest: Array<in String>, value: String) {  
    dest[0] = value    // only safe to write (consumer)  
}  
// Array<Any> works
```

“Use-site variance gives you flexibility exactly where needed.”

Star Projections – *

When you don't know or don't care about the type

Use * to say "any type".

```
val list: MutableList<*> = mutableListOf(1, "two", 3.0)
// list.add(4) // Error: can't add because type unknown
val first = list[0] // Type is Any?
```

Meaning of *

- ▶ For interface `Foo<out T> → Foo<*>` is like `Foo<out Any?>`
- ▶ For interface `Bar<in T> → Bar<*>` is like `Bar<in Nothing>`

Dart Equivalent

Dart uses dynamic or raw types (deprecated).
No direct star projection.

"Star projections are useful when type safety is not critical (legacy code, heterogeneous containers)."

Reified Type Parameters – inline and reified

The Problem

Due to type erasure (JVM), you normally cannot check a generic type at runtime:

```
fun <T> isA(value: Any) = value is T // Error
```

Solution: inline + reified

```
inline fun <reified T> isA(value: Any) = value is T
```

```
fun main() {  
    println(isA<String>("hello")) // true  
    println(isA<Int>("hello"))    // false  
}
```

The compiler inlines the function and substitutes the actual type.

Use Cases

- ▶ Type checks
- ▶ Creating instances of generic type (with `::class.java`)
- ▶ Reified type in lambdas

Dart variant

Dart has reified generics – you can use `is T` directly:

```
bool isA<T>(Object value) => value is T;
```

Exercise 1: Generic Function with Constraints

Task

Write a generic function `maxOf` that returns the maximum element from a list. The function should work for any type that implements `Comparable<T>`.

Function Signature

```
fun <T : Comparable<T>> maxOf(list: List<T>): T? {  
    // your code here  
}
```

Test with

```
println(maxOf(listOf(3, 7, 2, 9))) // 9  
println(maxOf(listOf("apple", "banana", "kiwi"))) // "kiwi"  
println(maxOf(emptyList())) // null
```

Hint: Use `fold` or manual iteration.

Exercise 2: Implement a Logger Using Delegation

Task

Create a simple logging system using delegation:

1. Define an interface `Logger` with a function `log(message: String)`.
2. Provide two implementations: `ConsoleLogger` (prints to console) and `FileLogger` (writes to a file – you can simulate with `println("File: $message")`).
3. Create a class `Application` that delegates logging to a `Logger` (use class delegation).

Sample Output

```
val app = Application(ConsoleLogger())
app.log("App started") // prints to console

val fileApp = Application(FileLogger())
fileApp.log("Error occurred") // "File: Error occurred"
```

"Delegation makes it easy to swap implementations without changing the class."

Quick Recap – Advanced Kotlin

Scope Functions

- ▶ `let`, `apply`, `run`, `with`, `also`
- ▶ Each has different receiver and return value
- ▶ Used for concise object configuration and transformations

Generics & Delegation

- ▶ Generic classes and functions
- ▶ Variance: `out` (covariant), `in` (contravariant)
- ▶ Type projections and star projections
- ▶ Class delegation (`by`) and property delegation (`lazy`, `observable`)

Project Milestone 4

Your UI prototype with navigation and ViewModel is due next week – we'll learn how to build it today.

“Now we apply Kotlin to build beautiful, reactive UIs with Jetpack Compose.”

Today's Roadmap – Building UIs with Compose

Part 5: Compose Fundamentals (30 min)

- ▶ What is Jetpack Compose?
- ▶ Composable functions
- ▶ Modifiers

Part 6: Layouts (25 min)

- ▶ Column, Row, Box
- ▶ Arrangement and alignment

Part 7: State Management (35 min)

- ▶ `mutableStateOf` and `remember`
- ▶ State hoisting
- ▶ Unidirectional data flow

Part 8: Material Design (20 min)

- ▶ Material 3 components
- ▶ Theming and typography

Part 9: Mini-Exercise (10 min)

“Leverage your Flutter knowledge – declarative UI patterns are the same!”

Jetpack Compose – Modern Android UI Toolkit

What is Jetpack Compose?

- ▶ Modern toolkit for building native Android UIs
- ▶ Declarative paradigm – UI is a function of state
- ▶ Write less code, no XML layouts
- ▶ Inspired by Flutter, SwiftUI, React

Flutter Analogy

Flutter uses 'Widget' trees; Compose uses '@Composable' functions.

- ▶ Flutter: 'StatelessWidget' → Compose: '@Composable' function
- ▶ Flutter: 'StatefulWidget' → Compose: 'mutableStateOf' + 'remember'

First Composable

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}
```

Just a function that describes UI.

"You already know declarative UI from Flutter – Compose feels similar!"

Composable Functions – The Building Blocks

Characteristics of Composables

- ▶ Annotated with '@Composable'
- ▶ Can call other composables
- ▶ Fast, can be re-executed frequently (recomposition)
- ▶ Should be side-effect free (idiomatically)

Example: A simple profile card

```
@Composable
fun ProfileCard(name: String, age: Int) {
    Column {
        Text(text = name, fontSize = 24.sp)
        Text(text = "Age: $age", color = Color.Gray)
    }
}
```

'Column' arranges children vertically.

"Think of composables as functions that emit UI – just like Flutter's 'build' method."

Modifiers – Styling and Layout Adjustments

What are Modifiers?

Modifiers decorate or add behavior to composables (padding, size, click, etc.). They are chainable.

```
@Composable
fun StyledText() {
    Text(
        text = "Hello",
        modifier = Modifier
            .padding(16.dp)
            .background(Color.Blue)
            .clickable { /* handle click */ }
    )
}
```

Common Modifiers

- ▶ padding, size, fillMaxWidth
- ▶ background, border
- ▶ clickable, scrollable

Flutter Analogy

Flutter uses properties on widgets; Compose uses modifier chain.

- ▶ Flutter: `Container(padding: ...)`
- ▶ Compose: `Modifier.padding(...)`

"Modifiers are the primary way to style and configure composables."

Basic Layouts – Arranging UI Elements

Column – Vertical arrangement

```
Column {  
    Text("Top")  
    Text("Middle")  
    Text("Bottom")  
}
```

Row – Horizontal

```
Row {  
    Text("Left")  
    Text("Right")  
}
```

Box – Stack

```
Box {  
    Image(...)  
    // background  
    Text("Overlay")  
    // foreground  
}
```

Flutter Equivalents

- ▶ Column → Column
- ▶ Row → Row
- ▶ Box → Stack

"Layouts in Compose are very similar to Flutter – easy to pick up!"

State in Compose – Making UI Reactive

State and Recomposition

When state changes, Compose recomposes affected parts.

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Clicked $count times")
    }
}
```

remember

Keeps value across recompositions (like `setState` in Flutter, but automatic).

mutableStateOf

Creates an observable value. Reading it subscribes to changes.

"State drives the UI – update state, UI updates automatically."

State Hoisting – Sharing and Testing State

What is State Hoisting?

Move state up to the caller, making the composable stateless and reusable.

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    Counter(count, { count++ })
}

@Composable
fun Counter(count: Int, onIncrement: () -> Unit) {
    Button(onClick = onIncrement) {
        Text("Clicked $count times")
    }
}
```

Benefits

- ▶ Reusable composables
- ▶ Easier to test
- ▶ Single source of truth

Flutter Analogy

Similar to lifting state up in Flutter.

Material Design 3 – Consistent, Beautiful UI

Material 3 Components

Compose provides ready-to-use Material Design components.

```
@Composable
fun MyScreen() {
    Button(onClick = {}) {
        Text("Click me")
    }
    Card {
        Text("Content inside a card")
    }
    TextField(value = "", onValueChange = {})
}
```

Theming

```
MaterialTheme(
    colorScheme = lightColorScheme(
        primary = Color.Blue,
        secondary = Color.Green
    ),
    typography = Typography()
) {
    // Your app content
```

Flutter Analogy

MaterialApp and ThemeData serve similar purpose.

"Material 3 gives your app a professional look with minimal effort."

Efficient Lists – LazyColumn and LazyRow

Why LazyColumn?

For long lists, use LazyColumn (vertical) or LazyRow (horizontal). They compose only visible items, saving memory and performance.

```
@Composable
fun NumberList(numbers: List<Int>) {
    LazyColumn {
        items(numbers) { number ->
            Text("Item: $number")
        }
    }
}
```

Customizing items

```
itemsIndexed(numbers) { index, item ->
    Text("[$index] $item")
}
```

Flutter Analogy

Similar to ListView.builder – only builds visible items.

“Use LazyColumn for efficient scrolling lists.”

Navigation in Compose

Setup

Add dependency: implementation

```
"androidx.navigation:navigation-compose:$version"
```

and NavController

```
val navController = rememberNavController()  
NavHost(navController, startDestination = "home") {  
    composable("home") { HomeScreen(navController) }  
    composable("details") { DetailsScreen() }  
}
```

Navigating

```
navController.navigate("details")
```

Flutter Analogy

Similar to `Navigator.pushNamed(context, "route")`.

"Navigation component handles back stack and deep links."

Navigation with Arguments

Define route with argument

```
NavHost(navController, startDestination = "home") {  
    composable("home") { HomeScreen(navController) }  
    composable(  
        "details/{itemId}",  
        arguments = listOf(navArgument("itemId") { type = NavType.IntType })  
    ) { backStackEntry ->  
        val itemId = backStackEntry.arguments?.getInt("itemId") ?: 0  
        DetailsScreen(itemId)  
    }  
}
```

Navigate with argument

```
navController.navigate("details/42")
```

Flutter Analogy

Pass arguments via `settings.arguments` in `onGenerateRoute`.

"Arguments let you pass data between screens type-safely."

ViewModel – Business Logic and State

ViewModel in Compose

Use `viewModel()` to get a `ViewModel` instance (requires `androidx.lifecycle:lifecycle-viewmodel-compose`).

```
class MyViewModel : ViewModel() {
    private val _uiState = mutableStateOf("Initial")
    val uiState: State<String> = _uiState

    fun updateState(new: String) {
        _uiState.value = new
    }
}

@Composable
fun MyScreen(viewModel: MyViewModel = viewModel()) {
    Text(viewModel.uiState.value)
    Button(onClick = { viewModel.updateState("Clicked") }) {
        Text("Click")
    }
}
```

“ViewModel survives configuration changes – perfect for state.”

Exercise: Product List with Details

Task

Build a simple app with two screens:

1. **ProductListScreen**: Shows a list of product names (hardcoded). Tapping an item navigates to details.
2. **ProductDetailScreen**: Displays the selected product name and an "Add to cart" button.

Hints

- ▶ Use `LazyColumn` with `items`.
- ▶ Pass product index or ID as navigation argument.
- ▶ Use `ViewModel` to hold the list and selected item state.

Project Milestone 5 Reminder

Due Tomorrow: Add navigation and `ViewModel` to your own app, following the pattern above.

"Practice makes perfect – this exercise mirrors your milestone."

Custom Composables – Encapsulate UI Logic

Why create custom composables?

- ▶ Reuse across screens
- ▶ Encapsulate styling and behavior
- ▶ Improve code readability

Example: Custom Button

```
@Composable
fun PrimaryButton(
    text: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Button(
        onClick = onClick,
        modifier = modifier.fillMaxWidth(),
        colors = ButtonDefaults.buttonColors(
            containerColor = MaterialTheme.colorScheme.primary
        )
    ) {
        Text(text)
    }
}

// Usage PrimaryButton("Submit") { /* handle submit */ }
```

Theming – Consistent Look and Feel

MaterialTheme

Compose provides MaterialTheme to define colors, typography, and shapes globally.

```
@Composable
fun MyApp(content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = lightColorScheme(
            primary = Color(0xFF6200EE),
            secondary = Color(0xFF03DAC6),
            background = Color.White
        ),
        typography = Typography(
            bodyLarge = TextStyle(
                fontFamily = FontFamily.Default,
                fontSize = 16.sp
            )
        ),
        shapes = Shapes(
            small = RoundedCornerShape(4.dp),
            medium = RoundedCornerShape(8.dp)
        )
    ) {
        content()
    }
}
```

Dark Mode – Adapting to System Settings

Dynamic color schemes

Use `isSystemInDarkTheme()` to detect system theme.

```
@Composable
fun MyApp(content: @Composable () -> Unit) {
    val colorScheme = if (isSystemInDarkTheme()) {
        darkColorScheme(
            primary = Color(0xFFBB86FC),
            secondary = Color(0xFF03DAC5)
        )
    } else {
        lightColorScheme(
            primary = Color(0xFF6200EE),
            secondary = Color(0xFF03DAC6)
        )
    }

    MaterialTheme(colorScheme = colorScheme) {
        content()
    }
}
```

"Supporting dark mode improves user experience and is easy with Compose."

Class 06 Wrap-Up – Jetpack Compose

Core Concepts

- ▶ `@Composable` functions
- ▶ Modifiers for styling
- ▶ Layouts: `Column`, `Row`, `Box`
- ▶ State: `mutableStateOf` + `remember`
- ▶ State hoisting

Advanced Topics

- ▶ Lazy lists: `LazyColumn`, `LazyRow`
- ▶ Navigation with arguments
- ▶ `ViewModel` integration
- ▶ Custom composables
- ▶ Theming and dark mode

Project Milestone 5 Reminder Due next Monday.

- ▶ Implement a Jetpack Compose UI for your project.
- ▶ Add a home screen.
- ▶ Add one screen with a form and validation for enrollment.
- ▶ Add one screen for login in.
- ▶ Add one screen with a list and navigation for main menu.
- ▶ Add navigation and `ViewModel` to your app.

"You now have the skills to build polished UIs – next we'll add data persistence!"

Coming Up – Storing Data Locally with Room

Why Room?

- ▶ SQLite abstraction layer
- ▶ Compile-time query verification
- ▶ Integrates with LiveData/Flow
- ▶ Handles database migrations

What we'll cover

- ▶ Entities and DAOs
- ▶ Database setup
- ▶ Type converters
- ▶ CRUD operations with coroutines
- ▶ Using Room with ViewModel and Flow